

Отчёт по прохождению 3 этапа внешнего курса

Продвинутые темы

Терентеевская Светлана Александровна

2026-05-15

Содержание I

- 1 Информация
- 2 Вводная часть
- 3 Теоретическое введение
- 4 Прохождение внешнего курса
- 5 Выводы
- 6 Список литературы

Раздел 1

Информация

- Терентеевская Светлана Александровна

- Терентеевская Светлана Александровна
- Группа НКАбд-05-25, студент бакалавра

- Терентеевская Светлана Александровна
- Группа НКАбд-05-25, студент бакалавра
- Российский университет дружбы народов им. П. Лумумбы

- Терентеевская Светлана Александровна
- Группа НКАбд-05-25, студент бакалавра
- Российский университет дружбы народов им. П. Лумумбы
- 1032253498@rudn.ru

- Терентеевская Светлана Александровна
- Группа НКАбд-05-25, студент бакалавра
- Российский университет дружбы народов им. П. Лумумбы
- 1032253498@rudn.ru
- <https://github.com/SvetlanaTerenteevskaya>

Раздел 2

Вводная часть

Цель работы

Пройти 3 этап курса «Введение в Linux» на платформе Stepik и ознакомиться с функционалом Linux.

Просмотреть видео и на основе полученной информации пройти тестовые и интерактивные задания.

Раздел 3

Теоретическое введение

Линукс - в части случаев GNU/Linux — семейство Unix-подобных операционных систем на базе ядра Linux, включающих тот или иной набор утилит и программ проекта GNU, и, возможно, другие компоненты. Как и ядро Linux, системы на его основе, как правило, создаются и распространяются в соответствии с моделью разработки свободного и открытого программного обеспечения. Linux-системы распространяются в основном бесплатно в виде различных дистрибутивов — в форме, готовой для установки и удобной для сопровождения и обновлений, — и имеющих свой набор системных и прикладных компонентов, как свободных, так и проприетарных.

Раздел 4

Прохождение внешнего курса

Прохождение внешнего курса

В vim команда выхода - :q (двоеточие для командной строки, q-quit, Enter - выполнить). Остальное: q-макрос, Q-режим Ez, Esc-смена режима, а не вывод (рис. 1).

Какую клавишу(и) нужно нажать на клавиатуре, чтобы выйти из редактора vim? Считайте, что вы только что открыли файл и вам сразу понадобилось выйти из редактора.

Выберите один вариант из списка

☒ Верно.

- ☐ "q", затем "Enter"
- ☐ "Esc"
- ☐ "Q"
- ☒ ": ", затем "q", затем "Enter"
- ☐ "q"

Следующий шаг

[Решить снова](#)

Рисунок 1: 1 задание

Последовательность любых символов кроме пробелов. Пробелов 4, значит 5 фрагментов (больших слов). W перепрыгнет через . (так как точка не пробел). w-9 нажатий, W-5, через 10 оба в конце. w - буквы, цифры, _ считаются, знаки .,=,!-отдельные слова (рис. 2).

А для того, чтобы убедиться, что вы разобрались, отметьте ниже **все верные** утверждения про следующую строку:

Примечание: во всех утверждениях имеется ввиду, что мы находимся в редакторе vim, включен нормальный режим работы и курсор находится в самом начале строки.

Подсказка: чтобы вызвать **vim-справку** по, например, перемещению **w**, нужно открыть vim и ввести команду **:help w**. Вы попадете в место справки, где описано это перемещение, а так как все перемещения описаны рядом, то двигаясь по тексту вверх и вниз можно прочитать и про **e** и про **b** и, самое главное, про **word** и **WORD**. Кроме того, можно вызвать сразу справку по термину **word** при помощи **:help word**. Чтобы закрыть справку, нужно ввести команду **:q**.

Выберите все подходящие ответы из списка

✔ Правильно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ В этой строке 5 "больших слов" (WORD)
- ☒ Нажимая только на W, нельзя переместить курсор на "."
- ☐ В этой строке 9 "больших слов" (WORD)
- ☒ После 10 нажатий на W курсор окажется там же, где бы он был после 10 нажатий на w
- ☒ В этой строке 9 "слов" (word)
- ☐ В этой строке 12 "слов" (word)

Следующий шаг

[Решить снова](#)

Рисунок 2: 2 задание

x2wwwPr - x удалит только o, затем 2w перейдет не туда. В итоге получится ne two three three three four five или подобное(рис. 3).

Предположим, что в текстовом файле записана одна единственная строка:

one two three four five

и вам нужно преобразовать её в строку

three four four four five

Какие(ой) из предложенных ниже наборов нажатий клавиш выполняют такое редактирование? В этих наборах нажатие на клавишу Esc обозначается как <Esc> (т.е. знаки "<" и ">" не несут отдельного смысла).

Примечание: во всех утверждениях имеется в виду, что мы находимся в редакторе vim, включен нормальный режим работы и курсор находится в самом начале строки.

Выберите все подходящие ответы из списка

✓ Абсолютно точно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ d2wwifour four <Esc>
- ☒ xxxxxxxxwywPp
- ☒ ddithree four four four five<Esc>
- ☒ d2w\$bifour four <Esc>
- ☐ x2wwyWpP
- ☒ d2wwyWpP

Следующий шаг

[Решить снова](#)

Рисунок 3: 3 задание

% - весь файл, s - замена, /Windows/Linux - что на что заменяем, без флага g
- только первое вхождение в строке (рис. 4).

Предположим, что вы открыли файл в редакторе vim и хотите заменить в этом файле все строки, содержащие слово `Windows`, на такие же строки, но со словом `Linux`. Если в какой-то строке слово `Windows` встречается больше, чем один раз, то заменить на `Linux` в этой строке нужно **только самое первое** из этих слов.

Какую команду нужно ввести для этого в vim? Укажите необходимую команду целиком (т.е. **включая** ввод ":" в самом начале), однако нажатие на `Enter` после ввода команды обозначать никак **не нужно**.

Напишите текст



Абсолютно точно.

:%s/Windows/Linux/

Следующий шаг

Решить снова

Рисунок 4: 4 задание


```
#!/bin/bash  
echo "Arguments are: \$1=$1 \$2=$2"
```

Построчно:

#!/bin/bash — интерпретатор
echo «Arguments are: \$1=\$1 \$2=\$2» — выводим текст, где \$1 даёт
буквально \$1, а \$1 подставляет первый аргумент (рис. 8).

Напишите скрипт на `bash`, который принимает на вход два аргумента и выводит на экран строку следующего вида:

Например, если ваш скрипт называется `./script.sh`, то при запуске его `./script.sh one two` на экране должно появиться:

а при запуске `./script.sh three four` будет:

Подсказка: в случае проблем с решением задачи, обратите внимание [на наши рекомендации по написанию скриптов](#).

Напишите программу. Тестируется через stdin → stdout

✔ Правильно.

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 #!bin/bash
2 echo "Arguments are: $1=$1 $2=$2"
3
4
5
6
7
```

Следующий шаг

[Решить снова](#)

Рисунок 8: 8 задание

Имя может начинаться с буквы или *, затем буквы, цифры,* 123variable - цифры в начале, остальные: /, -, пробел, \$\$, @ - недопустимы (рис. 9).

Какие из представленных ниже строк **могут** быть именами переменных в bash? Выберите **все** подходящие варианты!

Подсказка: если все варианты ответов являются неверными, то не отмечайте ни один из них и нажимайте кнопку "Отправить"/"Submit".

Выберите все подходящие ответы из списка

✔ Всё получилось!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ 123variable
- ☒ variable123
- ☐ vari/able
- ☐ var-i-able
- ☐ var i able
- ☐ variab\$\$le
- ☐ var@iable

Следующий шаг

[Решить снова](#)

Рисунок 9: 9 задание

!(4 ≤ 3): 4 ≤ 3 - ложно, отрицание и ложь получается истина. \$# -ge 0: \$#
- количество аргументов, переданных скрипту, оно всегда целое неотрицательное, а -ge 0 значит больше или равно 0, значит условие всегда истинно (рис. 10).

Вы можете скачать и изучить скрипт, который мы показали в видеофрагменте: branching1.sh.

Предположим, вы пишете скрипт на `bash` и хотите использовать в нем конструкцию `if` в следующем фрагменте:

```
if [[ ... ]]
then
  echo "True"
fi
```

Вы можете вписать вместо `"..."` (внутри `[[ ]]` и **не забудьте про пробелы** после `[[` и перед `]]`) любое из перечисленных ниже условий. Однако мы просим вас выбрать только те из них, при которых `echo` напечатает на экран `True` вне зависимости от того, с какими параметрами был запущен ваш скрипт и какие в нем есть переменные.

Например, условие `0 -eq 0` **подходит**, т.к. ноль всегда равен нулю вне зависимости от аргументов и переменных внутри скрипта и на экран будет напечатано `True`. В то же время условие `$var1 -eq 0` **не подходит**, так как в переменной `var1` как может быть записан ноль (тогда будет напечатано `True`), так его может и не быть (тогда ничего напечатано не будет).

Примечание: если вы планируете проверять варианты ответов у себя в терминале, обратите внимание на то, что содержащие символ `$` тексты могут изменяться при копировании — не забудьте отредактировать их в соответствии с изображением на экране. Это связано с особенностями написания `$` в некоторых видах заданий на Stepik.

Выберите все подходящие ответы из списка

☒ Прекрасный ответ.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ ! (4 -le 3)
- ☐ -z ""
- ☐ \$var1 == \$var2 && \$var1 != \$var2
- ☒ \$# -ge 0
- ☐ \$# -gt 0
- ☐ -n \$1

Следующий шаг

Решить снова

Рисунок 10: 10 задание

```
#!/bin/bash
case $1 in
  0) echo "No students" ;;
  1) echo "1 student" ;;
  2|3|4) echo "$1 students" ;;
  *) echo "A lot of students" ;;
esac
```

Построчно:

case \$1 in — проверяем первый аргумент

❶) echo «No students» ;; — если 0, вывести «No students»

2|3|4) echo «\$1 students» ;; — если 2, 3 или 4, вывести «X students»

*) echo «A lot of students» ;; — если любое другое число, вывести «A lot of students»

esac — конец конструкции case (рис. 11).

```
#!/bin/bash
case $1 in
  0) echo "No students" ;;
  1) echo "1 student" ;;
  2|3|4) echo "$1 students" ;;
  *) echo "A lot of students" ;;
esac
```

Построчно:

case \$1 in — проверяем первый аргумент

❶ echo «No students» ;; — если 0, вывести «No students»

❷ echo «1 student» ;; — если 1, вывести «1 student»

2|3|4) echo «\$1 students» ;; — если 2, 3 или 4, вывести «X students»

*) echo «A lot of students» ;; — если любое другое число, вывести «A lot of students»

esac — конец конструкции case (рис. 11).

Напишите скрипт на bash, который принимает на вход один аргумент (целое число от 0 до бесконечности), который будет обозначать число студентов в аудитории. В зависимости от значения числа нужно вывести разные сообщения.

Соответствие входа и выхода должно быть таким:

```
0 --> No students
1 --> 1 student
2 --> 2 students
3 --> 3 students
4 --> 4 students
5 и больше --> A lot of students
```

Примечание а): выводить нужно только строку справа, т.е. "-->" выводить не нужно.

Примечание б): в последней строке слово "lot" с маленькой буквы!

Примечание 2: в этой и всех последующих задачах на написание скриптов, если не указано явно, что нужно **проверять вход** (например, что он будет именно числом и именно от 0 до бесконечности), то этого делать **не нужно**!

Пример №1: если ваш скрипт называется `./script.sh`, то при запуске его как `./script.sh 1` на экране должно появиться:

```
1 student
```

Пример №2: если ваш скрипт называется `./script.sh`, то при запуске его как `./script.sh 5` на экране должно появиться:

```
A lot of students
```

Подсказка: в случае проблем с решением задачи, обратите внимание [на наши рекомендации по написанию скриптов](#).

Напишите программу. Тестируется через stdin → stdout

✓ Прекрасный ответ.

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 #!/bin/bash
2 case $1 in
3 0) echo "No students" ;;
4 1) echo "1 student" ;;
5 2|3|4) echo "$1 students" ;;
6 *) echo "A lot of students" ;;
7 esac
8
9
10
11
```

Рисунок 11: 11 задание

Оба значения (`var=3` и `var=5`) не подпадают ни под одно из условий (`-gt 5`, `-lt 3`, `-eq 4`), поэтому оба раза выполняется ветка `else`, печатающая `four` (рис. 12).

Посмотрите на фрагмент `bash`-скрипта:

```
if [[ $var -gt 5 ]]
then
    echo "one"
elif [[ $var -lt 3 ]]
then
    echo "two"
elif [[ $var -eq 4 ]]
then
    echo "three"
else
    echo "four"
fi
```

Какие строки и в какой последовательности он выведет на экран, если сначала этот скрипт запустили задав переменную **var=3**, а затем запустили еще раз, но уже с **var=5**.

Выберите один вариант из списка

☒ Верно.

- ☐ Сначала two, потом one
- ☐ Сначала one, потом two
- ☐ Сначала two, потом four
- ☒ Сначала four, потом four

Следующий шаг

[Решить снова](#)

Рисунок 12: 12 задание

```
while true; do
    echo "enter your name:"
    read name
    [ -z "$name" ] && echo "bye" && exit
    echo "enter your age:"
    read age
    [ "$age" -eq 0 ] && echo "bye" && exit
    if [ "$age" -le 16 ]; then group="child"
    elif [ "$age" -le 25 ]; then group="youth"
    else group="adult"
    fi
    echo "$name, your group is $group"
done
```

Построчно:

while true; do — бесконечный цикл

echo «enter your name:» — приглашение ввести имя

read name — читаем имя

[-z «\$name»] && echo «bye» && exit — если имя пустое, выводим «bye» и выходим

echo «enter your age:» — приглашение ввести возраст

read age — читаем возраст

[«\$age» -eq 0] && echo «bye» && exit — если возраст 0, выводим «bye» и выходим

if [«\$age» -le 16]; then group=«child» — если возраст ≤ 16 , группа child

elif [«\$age» -le 25]; then group=«youth» — иначе если ≤ 25 , группа

youthelse group=«adult» — иначе (старше 25) группа adult

echo «\$name, your group is \$group» — выводим результат

done — конец цикла(рис. 13, рис. 14).

Напишите скрипт на bash, который будет определять в какую возрастную группу попадают пользователи. При запуске скрипт должен вывести сообщение **"enter your name:"** и ждать от пользователя ввода имени (используйте `read`, чтобы прочитать его). Когда имя введено, то скрипт должен написать **"enter your age:"** и ждать ввода возраста (опять нужен `read`). Когда возраст введен, скрипт пишет на экран **"<Имя>, your group is <группа>"**, где **<группа>** определяется на основе возраста по следующим правилам:

- младше либо равно 16: **"child"**,
- от 17 до 25 (включительно): **"youth"**,
- старше 25: **"adult"**.

После этого скрипт опять выводит сообщение **"enter your name:"** и всё начинается по новой (бесконечный цикл!). Если в какой-то момент работы скрипта будет введено **пустое имя** или **возраст 0**, то скрипт должен написать на экран **"bye"** и закончить свою работу (выход из цикла!).

Примеры корректной работы скрипта:

№1

```
./script.sh
enter your name:
Egor
enter your age:
16
Egor, your group is child
enter your name:
Elena
enter your age:
0
bye
```

№2:

```
./script.sh
enter your name:
Elena Petrovna
enter your age:
25
Elena Petrovna, your group is youth
enter your name:

bye
```

Рисунок 13: 13 задание

✔ Всё правильно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

[Решить снова](#)

Следующий шаг

30 / 72

- -lt, (<) - меньше

3 не больше 5, 3 не меньше 3, 3 не равно 4. 5 не больше 5, 5 не меньше 3, 5 не равно 4. Оба раза выведет four.(рис. 15).

- -lt, (<) - меньше
- -gt - больше

3 не больше 5, 3 не меньше 3, 3 не равно 4. 5 не больше 5, 5 не меньше 3, 5 не равно 4. Оба раза выведет four.(рис. 15).

- -lt, (<) - меньше
- -gt - больше
- -eq - равно

3 не больше 5, 3 не меньше 3, 3 не равно 4. 5 не больше 5, 5 не меньше 3, 5 не равно 4. Оба раза выведет four.(рис. 15).

Посмотрите на фрагмент bash-скрипта:

```
for str in a , b , c_d
do
  echo "start"
  if [[ $str > "c" ]]
  then
    continue
  fi
  echo "finish"
done
```

Если запустить этот скрипт, то **сколько раз** на экран будет выведено слово **"start"**, а сколько раз слово **"finish"**?

Выберите один вариант из списка

☒ Всё правильно.

- ☐ 3 раза "start" и 3 раза "finish"
- ☐ 3 раза "start" и ни разу "finish"
- ☒ 5 раз "start" и 4 раза "finish"
- ☐ 5 раз "start" и ни разу "finish"

Следующий шаг

Решить снова

Рисунок 15: 14 задание

- let « $a=a+b$ » — классика.

Все варианты корректно прибавляют b к a (рис. 16).

- `let «a=a+b»` — классика.
- `let a=a+b` — без кавычек работает, если без пробелов.

Все варианты корректно прибавляют `b` к `a` (рис. 16).

- `let «a=a+b»` — классика.
- `let a=a+b` — без кавычек работает, если без пробелов.
- `let «a+=b»` — сокращённая запись.

Все варианты корректно прибавляют `b` к `a` (рис. 16).

- `let «a=a+b»` — классика.
- `let a=a+b` — без кавычек работает, если без пробелов.
- `let «a+=b»` — сокращённая запись.
- `let a=a+b` — с `$` тоже работает.

Все варианты корректно прибавляют `b` к `a` (рис. 16).

- `let «a=a+b»` — классика.
- `let a=a+b` — без кавычек работает, если без пробелов.
- `let «a+=b»` — сокращённая запись.
- `let a=a+b` — с `$` тоже работает.
- `let «a = a + b»` — пробелы допустимы в кавычках.

Все варианты корректно прибавляют `b` к `a` (рис. 16).

program; if [[\$? -eq 0]] — сохраняет код возврата. if program > some_file.txt — проверяет код, stdout в файл. Остальные пытаются использовать вывод вместо кода возврата (рис. 18).

Мы рассказали, что можно проверить код возврата внешней программы прямо в конструкции `if` при помощи `if `program options arguments`` (действия внутри `if` выполняются, если программа закончилась с кодом 0). Однако это не всегда правда! Если запуск внешней программы выводит что-то в `stdout`, то в проверку `if` поступит именно этот вывод, а не код возврата! Вы можете убедиться в этом, написав простой `bash`-скрипт с использованием, например, `if `pwd``.

Однако как быть, если хочется всё-таки запустить программу `program`, которая пишет что-то в `stdout` и потом выполнить какие-то действия если ее код возврата равен 0? Выберите **все верные** утверждения или правильно работающие конструкции `if`.

Примечание: во всех вариантах ответов, где есть кавычка, **используется именно косая кавычка (')**, а не обычная (") или двойная (").

Выберите все подходящие ответы из списка

Отлично!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Сначала `var=program`, затем `if [[ $var -eq 0 ]]`
- ☐ Ничего сделать нельзя
- ☐ `if [[ `program` -eq 0 ]]`
- ☒ Сначала запустить `program`, затем `if [[ $? -eq 0 ]]`
- ☒ `if `program` > some_file.txt``

Следующий шаг

[Решить снова](#)

Рисунок 18: 17 задание

```
#!/bin/bash
gcd() {
    if [ $1 -eq $2 ]; then echo "GCD is $1"; return
    elif [ $1 -gt $2 ]; then gcd $(( $1 - $2 )) $2
    else gcd $1 $(( $2 - $1 )); fi
}
while read m n; do
    [ -z "$m" ] && echo "bye" && exit
    gcd $m $n
done
```

Построчно:

`gcd()` { — объявление функции

`if [ $1 -eq $2 ]; then` — если числа равны

`echo «GCD is $1»` — выводим НОД

`return` — выходим из функции

`elif [ $1 -gt $2 ]; then` — если первое больше второго

`gcd $((($1 - $2)) $2` — вызываем `gcd` с разностью

`else` — иначе (второе больше первого)

`gcd $1 $((($2 - $1))` — вызываем `gcd` с разностью

`while read m n; do` — читаем два числа

`[ -z «$m» ] && echo «bye» && exit` — если пустая строка, выводим `bye` и выходим

`gcd $m $n` — вызываем функцию

`done` — конец цикла (рис. 19, рис. 20).

Напишите скрипт на `bash`, который будет искать наибольший общий делитель ([НОД](#), `greatest common divisor`, `GCD`) двух чисел. При запуске ваш скрипт не должен ничего писать на экран, а просто ждать ввода двух натуральных чисел через пробел (для этого можно использовать `read` и указать ему две переменные – см. пример в видеофрагменте). После ввода чисел скрипт считает их НОД и выводит на экран сообщение "**GCD is <посчитанное значение>**", например, для чисел 15 и 25 это будет "GCD is 5". После этого скрипт опять входит в режим ожидания двух натуральных чисел. Если в какой-то момент работы пользователь ввел вместо этого пустую строку, то нужно написать на экран **"bye"** и закончить свою работу.

Вычисление НОД несложно реализовать с помощью [алгоритма Евклида](#). Вам нужно написать функцию `gcd`, которая принимает на вход два аргумента (назовем их **M** и **N**). Если аргументы равны, то мы нашли НОД – он равен **M** (или **N**), нужно выводить соответствующее сообщение на экран (см. выше). Иначе нужно сравнить аргументы между собой. Если **M больше N**, то запускаем ту же функцию `gcd`, но в качестве первого аргумента передаем **(M-N)**, а в качестве второго **N**. Если же наоборот, **M меньше N**, то запускаем функцию `gcd` с первым аргументом **M**, а вторым **(N-M)**.

Пример корректной работы скрипта:

```
./script.sh
10 15
GCD is 5
7 3
GCD is 1
bye
```

Примечание: в вызове функции из себя самой нет ничего страшного или неправильного, т.ч. смело вызывайте `gcd` прямо внутри `gcd` !

Примечание 2: для завершения работы функции в произвольном месте, можно использовать инструкцию `return` (все инструкции функции после `return` выполняться не будут). В отличие от `exit` эта команда завершит только функцию, а не выполнение всего скрипта целиком. Однако в данной задаче можно обойтись и без использования `return`!

Рисунок 19: 18 задание

Напишите программу. Тестируется через stdin → stdout

✓ Хорошие новости, верно!

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 #!/bin/bash
2 gcd() {
3     if [ $1 -eq $2 ]; then echo "GCD is $1"; return
4     elif [ $1 -gt $2 ]; then gcd $((($1-$2)) $2
5     else gcd $1 $((($2-$1))); fi
6 }
7 while read m n; do
8     [ -z "$m" ] && echo "bye" && exit
9     gcd $m $n
10 done
```

Следующий шаг

Решить снова

Вы получили: 4 балла

Верно решили 28 476 учащихся

Рисунок 20: 18 задание

c1 — локальная, глобальная остаётся пустой (выводит пусто). c2 — глобальная, сумма $2 \times (1 + 2 + \dots + 10) = 110$ (рис. 21).

Посмотрите на функцию из bash-скрипта:

```
counter () # takes one argument
{
    local let "c1+=${1}"
    let "c2+=${1}*2"
}
```

Впишите в форму ниже **строку**, которую выведет на экран команда `echo "counters are $c1 and $c2"` если она находится в скрипте **после десяти вызовов** функции `counter` с параметрами сначала 1, затем 2, затем 3 и т.д., последний вызов с параметром 10.

Подсказка: этот пример можно решить в уме, но если система проверки не принимает ваше решение, то возможно вы что-то упустили (возможно что-то совсем небольшое/невидимое 🤔). В этом случае имеет смысл написать небольшой скрипт на bash, который проделает ровно то, что указано в задании и посимвольно сверить свой ответ с тем, что он выдаст на экран.

Напишите текст



Так точно!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

counters are and 110

Следующий шаг

Решить снова

Рисунок 21: 19 задание

```
#!/bin/bash
while true;
do
    read a op b
    if [[ $a == "exit" ]];
    then
        echo "bye"
        exit
    elif [[ ! $a =~ ^[0-9]+$ || ! $b =~ ^[0-9]+$ ]];
    then
        echo "error"
        exit
    else
        case $op in
            "+") let "res = a + b" ;;
            "-") let "res = a - b" ;;
            "*") let "res = a * b" ;;
```

```
"/") let "res = a / b" ;;  
"%") let "res = a % b" ;;  
"**") let "res = a ** b" ;;  
*) echo "error"; exit ;;  
esac  
echo "$res"  
fi  
done
```

while true; do — бесконечный цикл
 read a op b — читаем число, операцию, число
 if [[\$a == «exit»]]; then — если введено «exit»
 echo «bye» — выводим bye, exit — выходим
 elif [[! a = [0-9]+ || ! b = [0-9]+]]; then — если a или b не числа
 echo «error»; exit — выводим error и выходим
 case \$op in
 «+») let «res = a + b» ;; — сложение
 «-») let «res = a - b» ;; — вычитание
 «*») let «res = a * b» ;; — умножение
 «/») let «res = a / b» ;; — деление
 «%») let «res = a % b» ;; — остаток
 «^») let «res = a ^ b» ;; — возведение в степень
 *) echo «error»; exit ;; — неизвестная операция → error
 esac — конец case
 echo «\$res» — выводим результат
 done — конец цикла (рис. 22, рис. 23).

Напишите **калькулятор** на bash. При запуске ваш скрипт должен ожидать ввода пользователем команды (при этом на экран выводить ничего не нужно). Команды могут быть трех типов:

1. Слово **"exit"**. В этом случае скрипт должен вывести на экран слово "bye" и завершить работу.
2. **Три аргумента через пробел** -- первый операнд (целое число), операция (одна из "+", "-", "*", "/", "%", "**") и второй операнд (целое число). В этом случае нужно произвести указанную операцию над заданными числами и вывести результат на экран. После этого переходим в режим ожидания новой команды.
3. **Любая другая команда** из одного аргумента или из трех аргументов, но с операцией не из списка. В этом случае нужно вывести на экран слово **"error"** и завершить работу.

Чтобы проверить работу скрипта, вы можете записать сразу несколько команд в файл и передать его скрипту на stdin (т.е. выполни `./script.sh < input.txt`). В этом случае он должен вывести сразу все ответы на экран.

Рисунок 22: 20 задание

Напишите программу. Тестируется через stdin → stdout

✓ Хорошие новости, верно!

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```

1  #!/bin/bash
2
3  while true
4  do
5      read a op b
6      if [[ $a == "exit" ]]
7      then
8          echo "bye"
9          exit
10     elif [[ ! $a =~ ^[0-9]+$ || ! $b =~ ^[0-9]+$ ]]
11     then
12         echo "error"
13         exit
14     else
15         case $op in
16             "+") let "res = a + b" ;;
17             "-") let "res = a - b" ;;
18             "/" let "res = a / b" ;;
19             "*" let "res = a * b" ;;
20             "%" let "res = a % b" ;;
21             "**") let "res = a ** b" ;;
22             *) echo "error"; exit ;;
23         esac
24         echo "$res"
25     fi
26 done

```

Следующий шаг

[Решить снова](#)

Рисунок 23: 20 задание

-name ищет только имя, -path — весь путь. Если файл с нужным именем лежит в подпапке, -path может найти его, а -name нет.(рис. 25).

Задание на понимание работы опций `-path` и `-name` команды `find`. Отметьте **все верные** утверждения из перечисленных ниже.

Выберите все подходящие ответы из списка

✓ Так точно!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Опция -path используется только для поиска директорий, а -name только для поиска файлов
- ☐ Опции -path и -name всегда работают одинаково
- ☐ Опция -path аналогична -name, но игнорирует размер букв (строчные/прописные) в имени файла
- ☐ Если заменить в команде поиска -name, на -path, то результат поиска всегда останется неизменным
- ☒ В некоторых случаях find с -name найдет меньше файлов, чем find с таким же запросом, но с -path

Следующий шаг

[Решить снова](#)

Рисунок 25: 22 задание

file1 — глубина 2. file2 — глубина 3. file3 — глубина 4 (не попадает в -maxdepth 3) (рис. 26).

Предположим, что в директории `/home/bi/` есть следующая структура файлов и поддиректорий:

```
/home/bi/  
├── dir1  
│   ├── file1  
│   └── dir2  
│       ├── file2  
│       └── dir3  
│           └── file3
```

Какие(ой) из трех файлов (`file1`, `file2`, `file3`) будут найдены по команде `find /home/bi -mindepth 2 -maxdepth 3 -name "file*" ?`

Выберите один вариант из списка

☒ Отлично!

- ☒ Все кроме file3
- ☐ Все кроме file2
- ☐ Только file1
- ☐ Только file3
- ☐ Только file2

Следующий шаг

Решить снова

Рисунок 26: 23 задание

Для файла из 10 строк, где слово есть в каждой, -A 1, -B 1, -C 1 из-за границ файла не добавляют дополнительных строк. Все команды выведут 10 строк → размер одинаков.(рис. 27).

Задание на понимание работы опций -A, -B и -C команды grep. Пусть у вас есть файл file.txt из 10 строк, причем в каждой строке есть слово "word". Если вы выполните на этом файле команды:

```
grep "word" file.txt > results.txt
grep -A 1 "word" file.txt > results.txt
grep -B 1 "word" file.txt > results.txt
grep -C 1 "word" file.txt > results.txt
```

то какая(ие) из них создаст файл results.txt наибольшего размера?

Выберите один вариант из списка

☒ Всё правильно.

- ☐ grep -C 1 "word" file.txt > results.txt
- ☐ grep -A 1 "word" file.txt > results.txt и grep -B 1 "word" file.txt > results.txt
- ☐ Все, кроме grep "word" file.txt > results.txt
- ☐ grep -A 1 "word" file.txt > results.txt
- ☒ results.txt будет одинакового размера во всех случаях

Следующий шаг

Решить снова

Рисунок 27: 24 задание

Шаблон: любой символ из xklXKL (необязательно), затем ubuntu в конце строки.

- Lubuntu — нет L в скобках, но Ubuntu подходит.

Uuuubuntu! — не подходит из-за ! (конец строки) (рис. 28).

Шаблон: любой символ из xklXKL (необязательно), затем ubuntu в конце строки.

- Lubuntu — нет L в скобках, но Ubuntu подходит.
- Linux...Ubuntu — Ubuntu в конце подходит.

Uuuubuntu! — не подходит из-за ! (конец строки) (рис. 28).

Шаблон: любой символ из xklXKL (необязательно), затем ubuntu в конце строки.

- Lubuntu — нет L в скобках, но Ubuntu подходит.
- Linux...Ubuntu — Ubuntu в конце подходит.
- Kubuntu — нет K в скобках, но ubuntu подходит.

Uuuubuntu! — не подходит из-за ! (конец строки) (рис. 28).

Шаблон: любой символ из xklXKL (необязательно), затем ubuntu в конце строки.

- Lubuntu — нет L в скобках, но Ubuntu подходит.
- Linux...Ubuntu — Ubuntu в конце подходит.
- Kubuntu — нет K в скобках, но ubuntu подходит.
- xubuntu — x есть в скобках.

Uuuubuntu! — не подходит из-за ! (конец строки) (рис. 28).

Предположим, что в файле `text.txt` записаны строки, показанные среди вариантов ответа. Отметьте только те из них, которые выведет на экран команда `grep -E "[xk\XKL]?[uU]buntu$" text.txt`.

Выберите все подходящие ответы из списка

✔ Всё правильно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Uuuubuntu!
- ☒ Lubuntu is better than Ubuntu
- ☒ Linux is not always Ubuntu
- ☒ I prefer Kubuntu
- ☒ The best OS is Xubuntu
- ☐ Well, xubuntu is OK

Следующий шаг

[Решить снова](#)

Рисунок 28: 25 задание

Без -n sed печатает каждую строку автоматически. Команда p печатает ещё раз. Итого 2 раза.(рис. 29).

Что произойдет, если в команде `sed -n "/[a-z]*/p" text.txt` не указывать опцию `-n` ?

Выберите один вариант из списка



Так точно!

- ☐ Появится сообщение об ошибке
- ☐ На экран будет выведено всё содержимое файла text.txt
- ☒ Каждая строчка будет выведена два раза
- ☐ На экран ничего не напечатается

Следующий шаг

Решить снова

Вы получили: 1 балл

Верно реши

Рисунок 29: 26 задание

Тут [A-Z] — большие буквы, {2,} — от 2 и более, пробел после — окружение справа. Замена с пробелом сохраняет форматирование (рис. 30).

Запишите в форму ниже инструкцию `sed`, которая заменит все "аббревиатуры" в файле `input.txt` на слово "abbreviation" и запишет результат в файл `edited.txt` (на экран при этом ничего выводить не нужно). Обратите внимание, что в инструкции должны быть указаны и сам `sed`, и оба файла!

Под "аббревиатурой" будем понимать слово, которое удовлетворяет следующим условиям:

- состоит только из больших букв латинского алфавита,
- состоит из хотя бы двух букв,
- окружено одним пробелом с каждой стороны.

При этом будем считать, что в тексте **не может быть две "аббревиатуры" подряд**. Например, текст `" YOU YOU and YOU!"` является **некорректным** (в нем есть две "аббревиатуры", но они идут подряд) и на таких примерах мы проверять вашу инструкцию **не будем**.

Пример: если у вас был текст `"Hi, I heard these songs by ABBA, TLA and DM I"`, то он должен быть преобразован в `"Hi, I heard these songs by ABBA, abbreviation and abbreviation I"`.

Примечание: после вашей замены "аббревиатуры" на слово "abbreviation" количество пробелов в тексте **не должно меняться!**

Внимание! Во время проверки **мы не запускаем команду**, которую вы ввели на реальном файле с "аббревиатурами" (это небезопасно, можно же ввести `rm -rf /*`)! Вместо этого мы сперва анализируем структуру вашей инструкции (например, что в ней использован именно `sed` и сделано это ровно один раз, что на вход подается `input.txt`, а результат будет записан в `edited.txt` и т.д.), а затем **запускаем её смысловую часть** (т.е. поиск по регулярному выражению и замена на "abbreviation") на тестовых примерах. К сожалению, наш запуск не идеально повторяет `sed`, но он очень близок к нему. Главная "несовместимость" заключается в том, что наша проверка не понимает идущие подряд символы, отвечающие за количество повторений (т.е. *, +, ? и {}). Однако эту "несовместимость" легко исправить указав при помощи "(" и ")" какой из символов к чему относится! Например, регулярное выражения `a+?` (ноль или один раз по одной или более букве "a") нужно записать как `(a+)?` (при этом запись `(a)+?`, конечно же, не поможет).

Напишите текст

☒ Верно.

```
sed 's/[A-Z]\{2,\} /abbreviation /g' input.txt > edited.txt
```

Следующий шаг

Решить снова

Рисунок 30: 27 задание

Опция `-persist` оставляет окна графиков после выхода из `gnuplot` (рис. 31).

Какую опцию нужно указать при запуске `gnuplot`, чтобы при его закрытии не были автоматически закрыты и все нарисованные в нём графики?

Выберите один вариант из списка

☒ Отличное решение!

- ☐ Графики и так не закрываются автоматически при закрытии `gnuplot`!
- ☐ Такой опции не существует
- ☐ `-s, --show-plots-after-exit`
- ☒ `-p, --persist`

Следующий шаг

Решить снова

Рисунок 31: 28 задание

`set xtics` — устанавливает деления на оси X
(«point 1, value».x1) x1 — метка «point 1, value » в координате x1
, — разделитель
(«point 2, value».x2) x2 — метка в координате x2
(«point 3, value».x3) x3 — метка в координате x3
. — конкатенация (склеивание строк) (рис. 33).

Предположим, что вы пишете gplot-скрипт и у вас в нем есть три переменные `x1`, `x2`, `x3`, в которых записаны координаты важных точек по оси OX (по возрастанию). Вы хотите, чтобы на этой оси было только три деления (т.е. три черточки) в этих самых координатах, а подписи этих делений были оформлены в виде **"point <номер точки>, value <значение соответствующей переменной>".**

Например, для `x1=0`, `x2=10`, `x3=20`, это были бы надписи "point 1, value 0" в точке с координатой 0 по горизонтали, "point 2, value 10" в точке с координатой 10 и "point 3, value 20" в точке с координатой 20.

Или, например, `x1=100`, `x2=150`, `x3=250`, это были бы надписи "point 1, value 100" в точке с координатой 100, "point 2, value 150" в точке с координатой 150 и "point 3, value 250" в точке с координатой 250.

Впишите в форму ниже **одну команду** (т.е. одну строку), которую нужно добавить в скрипт, для выполнения этой задачи.

Примечание: проверять, что переменные `x1`, `x2`, `x3` идут по возрастанию или что они являются числами **не нужно!**

Примечание 2: в видеофрагменте на предыдущем шаге звучал термин *конкатенация*, который важен для выполнения данного задания. Под *конкатенацией* обычно понимают "склеивание" двух строк в одну длинную строку, например, конкатенация строк "Данные из файла " и "data.csv" даст строку "Данные из файла data.csv".

Подсказка: настоятельно рекомендуем изучить примеры скриптов – в них есть большая часть решения!

Напишите текст

✓ Отличное решение!

```
set xtics ("point 1, value ".x1 x1, "point2, value ".x2 x2, "point3, value ".x3 x3)
```

Следующий шаг

Решить снова

Рисунок 33: 30 задание

Правильный текст:

```
text a=a+1 zrot=(zrot-20)%360 set view xrot,zrot splot -(x2+y2) pause 0.2 if  
(a<50) reread
```

Пояснение:

- $-(x^2+y^2)$ — отражение по горизонтали (Z).

Правильный текст:

```
text a=a+1 zrot=(zrot-20)%360 set view xrot,zrot splot -(x2+y2) pause 0.2 if  
(a<50) reread
```

Пояснение:

- $-(x^2+y^2)$ — отражение по горизонтали (Z).
- -20 вместо +10 — обратное вращение и ускорение (20 — в 2 раза быстрее 10) рис. 34).

Если вы не скачали на предыдущем шаге файлы [animated.gnu](#) и [move.rot](#), то скачайте их теперь, т.к. они понадобятся для выполнения задания.

Указанные файлы использовались в последнем видеофрагменте для создания вращающегося графика. Измените инструкции в файле `move.rot` (т.е. **добавлять** и **удалять** инструкции **нельзя!**) таким образом, чтобы:

- График **отразился зеркально** относительно горизонтальной поверхности. То есть там, где была точка $(10, 10, 200)$, станет точка $(10, 10, -200)$, где была точка $(-10, -10, 200)$ станет $(-10, -10, -200)$ и т.д. При этом точка $(0, 0, 0)$ останется на месте.
- Изображение стало **вращаться в обратную сторону**. То есть если раньше вращалось "влево", то теперь станет "вправо".
- Вращение стало **в два раза быстрее**. То есть станет в два раза больше перерисовок графика на каждую секунду вращения.

Измененный файл загрузите в форму ниже.

Примечание: наша система проверки **не может** запустить на вашем файле `move.rot` программу `gnuplot` и сравнить полученный график с заданным. Вместо этого **мы анализируем команды**, которые вы указали в файле. Поэтому если вы видите, что ваш скрипт в `gnuplot` работает точно по условию, а мы отвечаем "Incorrect/Неверно", то попробуйте упростить свою модификацию `move.rot` и отправить его еще раз.

Напишите текст



Отличное решение!

move.rot2.txt (88 bytes)

Следующий шаг

Решить снова

Рисунок 34: 31 задание

$a+wx$ добавляет всем wx , затем убирает лишнее у группы и остальных, добивается нужных прав. $u+wx$ добавляет x владельцу и w группе. $764=rwxrw-r-$ напрямую. $ug+w$ добавляет w владельцу и группе, x - владельцу и получается $rwxr-r-$. В остальных 467 - дает $r-rw-rwx$ (не то) и 777 - дает полные права всем (рис. 35).

Примечание: запись вида команда1; команда2; команда3 означает, что в терминале последовательно были выполнены все три команды (сначала команда1, затем команда2 и, наконец, команда3).

✔ Всё получилось!

- ☒ `chmod a+wx file.txt; chmod o-wx file.txt; chmod g-x file.txt`
- ☒ `chmod u+wx file.txt; chmod g+w file.txt`
- ☒ `chmod 764 file.txt`
- ☐ `chmod 467 file.txt`
- ☒ `chmod ug+w file.txt; chmod u+x file.txt`
- ☐ `chmod 777 file.txt`

[Решить снова](#)

Рисунок 35: 32 задание

- (sudo chmod a+w dir) — даёт запись всем, включая user.

- (sudo chmod a+w dir) — даёт запись всем, включая user.
- (sudo chown user.group dir) — user становится владельцем → может писать.

- (sudo chmod a+w dir) — даёт запись всем, включая user.
- (sudo chown user.group dir) — user становится владельцем → может писать.
- (sudo chown user dir) — user владелец → может писать.

- (sudo chmod a+w dir) — даёт запись всем, включая user.
- (sudo chown user.group dir) — user становится владельцем → может писать.
- (sudo chown user dir) — user владелец → может писать.
- sudo chmod o+x — даёт выполнение, не запись.

- `(sudo chmod a+w dir)` — даёт запись всем, включая user.
- `(sudo chown user.group dir)` — user становится владельцем → может писать.
- `(sudo chown user dir)` — user владелец → может писать.
- `sudo chmod o+x` — даёт выполнение, не запись.
- `chmod o+w` (без sudo) — нет прав менять.

- `(sudo chmod a+w dir)` — даёт запись всем, включая user.
- `(sudo chown user.group dir)` — user становится владельцем → может писать.
- `(sudo chown user dir)` — user владелец → может писать.
- `sudo chmod o+x` — даёт выполнение, не запись.
- `chmod o+w` (без sudo) — нет прав менять.
- `sudo chmod g+w` — группа root, а user в ней не состоит (рис. 36).

wc умеет считать: количество строк (-l), количество слов (-w), количество символов (-m) или байт (-c). Количество предложений и определенных букв не умеет (рис. 37).

Отметьте какие характеристики файла можно посчитать с использованием команды `wc`.

Выберите все подходящие ответы из списка

✓ Хорошая работа.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ Количество слов
- ☒ Количество строк
- ☐ Количество предложений
- ☐ Количество определенных букв (например, количество букв "А")
- ☒ Количество символов

Следующий шаг

Решить снова

Рисунок 37: 34 задание

du - показывает размер файлов и директории. -h - выводит размер в удобном формате. -s - показывает только общий размер текущей директории, без перечисления вложенных файлов и поддиректорий (рис. 38).

Впишите в форму ниже команду, которая выведет сколько места на диске занимает текущая директория (при этом **размер** нужно вывести **в удобном для чтения формате** (например, вместо 2048 байт надо выводить 2.0K) и **больше** на экран выводить **ничего не** нужно). В команде указывайте **только необходимые** для выполнения задания **опции и аргументы**, лишних опций указывать не нужно!

Пример: если в текущей директории есть два файла по 800 Кбайт и две поддиректории в каждой из которой лежит по файлу в 400 Кбайт, то загаданная команда должна вывести на экран одно число: 2.4M (также на экране может быть выведен еще и символ " ", обозначающий, что это размер именно *текущей* директории).

Напишите текст



Прекрасный ответ.

```
du -h -s
```

Следующий шаг

Решить снова

Рисунок 38: 35 задание

`mkdir` - создает директории. `{1..3}` - фигурные скобки раскрываются в `1 2 3`.
`dir{1..3}` превращается в `dir1 dir 2 dir 3` (рис. 39).

Впишите в форму ниже максимально короткую команду (т.е. в которой минимально возможное число символов), которая позволит создать в текущей директории 3 поддиректории с именами `dir1`, `dir2`, `dir3`.

Если вы придумали команду, которая выполняет эту задачу, а система проверки сообщает вам "Incorrect"/"Неверно", то скорее всего вы придумали не самую короткую команду из возможных!

Напишите текст



Верно. Так держать!

`mkdir dir{1..3}`

Следующий шаг

Решить снова

Рисунок 39: 36 задание

Я получила сертификат о прохождении курса (рис. 40).

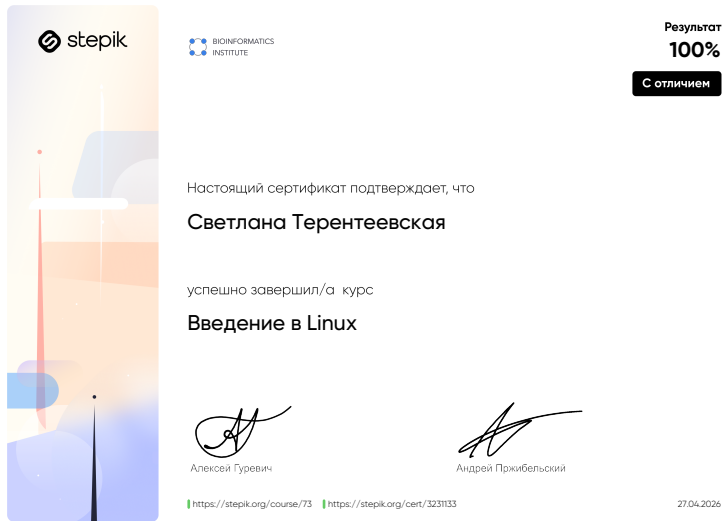


Рисунок 40: Сертификат

Раздел 5

Выводы

Я успешно прошла 3 этап курса «Введение в Linux» на платформе Stepik и ознакомилась с функционалом Linux, а также получила сертификат об окончании курса.

Раздел 6

Список литературы

Введение в Linux